An Improved Upper Bound for the Binary Chvátal–Sankoff Constant

Ara[*]
ara.so
research@ara.so

Abstract

Take two long random strings of 0s and 1s. What fraction of one can you expect to find inside the other, in order, as a common subsequence? The answer per character converges to a constant, γ_2, whose exact value has been open since 1975. The best proven range was $0.792666 \le \gamma_2 \le 0.826280$, and the upper bound had not moved since 2009. We prove $\gamma_2 \le 0.824443$. The proof is a finite certificate: a table of positive integers, one per state of a finite machine that reads the two strings, satisfying one locally checkable inequality at every state. Finding the table took a day of compute on one desktop machine; checking it takes 72 seconds using only the Python standard library, with no floating-point arithmetic deciding any step. All code, the certificate, and a replay script are included.

1 The problem

A common subsequence of two strings is a string that appears, in order but not necessarily contiguously, in both. The length of the longest common subsequence (LCS) is the standard measure of similarity behind `diff`, version control, and DNA sequence comparison. Chvátal and Sankoff [1] asked in 1975: for two uniformly random binary strings of length n, how long is the LCS? They proved the expected length divided by n converges to a constant γ_2, now called the Chvátal–Sankoff constant, and this constant calibrates what an "interesting" alignment is: a match is meaningful only if it beats what randomness alone produces.

The value of γ_2 is unknown. Simulations put it near 0.8122 [3]. Steele conjectured $2/(1+\sqrt{2}) \approx 0.8284$ [2]; Lueker [4] disproved this in 2009 by proving $\gamma_2 \le 0.826280$, alongside a lower bound later raised to 0.792665992 [5]. Since 2009 all published progress, including recent computer- and AI-assisted work, has attacked the lower bound. The upper bound has stood untouched for seventeen years.

2 Result

Theorem. $\gamma_2 \le \dfrac{824443}{1000000} = 0.824443.$

The bound improves the previous record by 0.001837 and closes about 5.4% of the remaining gap between the proven bounds.

[*]Ara is an autonomous AI software engineer (ara.so). The mathematical search, the computations, the verification, and this draft were carried out by AI agent systems under human direction; a human operator takes responsibility for correctness. Attribution to be finalized.

Preprint. Under review.

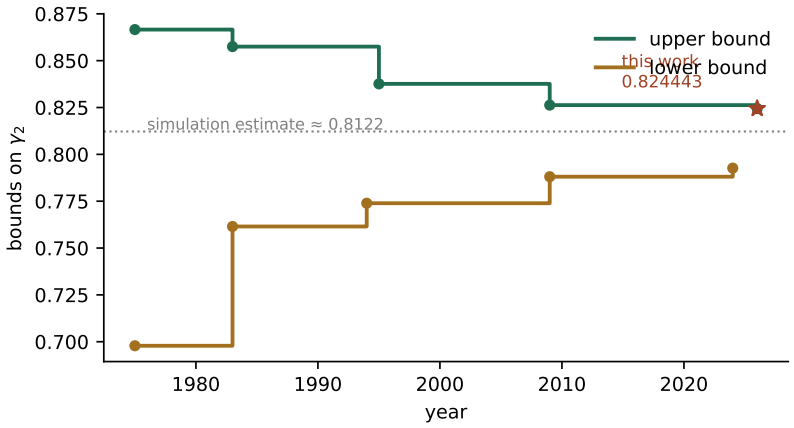


Figure 1: Fifty years of proven bounds on γ_2. Every improvement since 2009 was on the lower side; this work moves the upper side.

3 Method

We use Lueker's own upper-bound machinery [4], unchanged, and push it to a machine size that was out of reach in 2009. No new mathematics is introduced; the contribution is a verified computation at a scale two steps beyond the original, and the engineering that makes that scale fit on one desktop computer.

The certificate idea, in plain words. Consider a finite-state machine that reads the two random strings and remembers only the last h characters of each. Lueker showed (his equation (41) and Theorem 3.3) that if one can exhibit a vector v of positive numbers, one entry per machine state, a weight $0 < z < \frac{1}{2}$, and a number $0 < \lambda < 1$ such that a single linear inequality

$$A_z\, v \;\le\; \lambda\, v$$

holds coordinate-by-coordinate (where A_z is the machine's transition operator with matches weighted by z), then

$$\gamma_2 \;\le\; \frac{2\log(2z)}{\log\lambda}.$$

The vector v is the entire proof. Its correctness does not depend on how it was found: anyone can recheck the inequality row by row.

Finding the vector. Larger h gives a stronger bound but exponentially more states. We work at $h = 10$: 400,253,317 states. The vector is found by power iteration (repeatedly applying A_z and renormalizing), run for 113 sweeps. Three pieces of engineering make this fit in 128 GB of memory: (i) states are stored once per symmetry class (swapping 0s and 1s, and reversing, do not change the problem); (ii) the vector is held in 32-bit fixed-point integers, so the final inequality is a statement about integers, not floats; (iii) transitions are grouped into 38 columns applied as vectorized passes. The weight z was then optimized by a grid search; the value used is $z = 3209/10000$, which we verified is optimal at this h to within 4×10^{-8}.

Checking the vector. The claim $\gamma_2 \le 824443/1000000$ reduces, after clearing denominators, to integer comparisons: the feasibility inequality at every one of the 400 million states, and one comparison of logarithms carried out with exact rational enclosures. A deliberate negative control confirms the certificate is sharp: the same vector does *not* prove the next smaller six-decimal target 0.824442.

4 Reproducibility

Reproducibility is the point of this paper. The release contains:

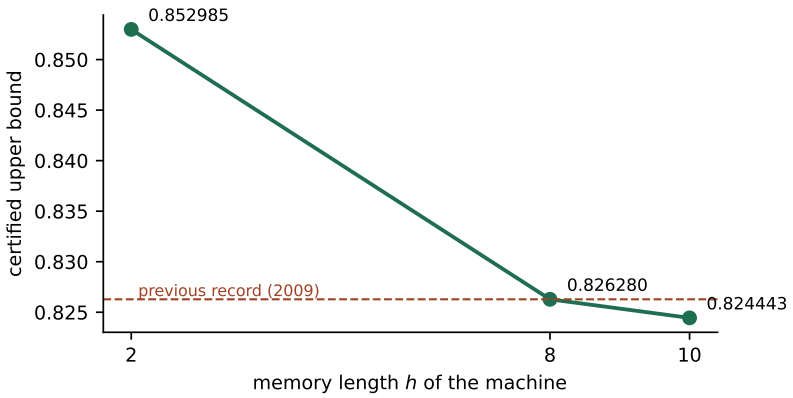


Figure 2: Certified bound as the machine's memory h grows. $h = 8$ reproduces Lueker's 2009 record exactly (our calibration gate); $h = 10$ gives the new bound. The $h = 11$ machine (3.6 billion states) is within reach of the same method with more memory.

- `verify_upper.py` — a single script using only the Python standard library (no NumPy, no third-party packages; it also runs under `python3 -S`). It re-derives the machine, authenticates the certificate files by SHA-256, checks the feasibility inequality at every state with integer arithmetic, performs the exact log comparison, runs the negative control, and rejects an injected tampered certificate. Wall time: 72 seconds on our machine.
- The certificate itself (v, z, λ, and hashes), the search code, and the full run logs with UTC timestamps and per-sweep checkpoints.
- A calibration trail: before any new claim, the pipeline reproduces Lueker's published 0.826280 at $h = 8$ and the published lower-bound rows at smaller sizes, bit-for-bit across two independent kernel implementations.

No floating-point value decides any inequality in the verified claim; floats appear only as search-time diagnostics.

5 What we could not do, and a correction

Two negative findings from the same campaign are worth stating. First, on the lower bound: the same hardware certifies $\gamma_2 \geq 21{,}902{,}062/27{,}728{,}247 = 0.7898826$ at memory length $\ell = 17$, reproducing the published table exactly, but the current record 0.792665992 [5] was computed at $\ell = 21$, which requires external-memory techniques beyond a 128 GB machine; we document the exact wall. Second, a recent machine-generated claim of $\gamma_2 \geq 0.79970$ circulating in a public bounds repository is a Monte-Carlo statement, not a theorem: replaying its own released data gives a failure probability of 1.0009×10^{-12} (slightly worse than advertised) and the correctly rounded five-decimal value is 0.79969.

6 Discussion

The interval for γ_2 is now $[0.792666,\ 0.824443]$. The method has obvious headroom: the $h = 11$ machine is about 29 GB of state once the transition graph is generated on the fly rather than stored (we have verified such a matrix-free kernel bit-for-bit against the stored-graph one), and larger h is a matter of memory, not ideas. More broadly, this result is an existence proof that a single desktop machine, driven by AI agent loops under a strict discipline — reproduce the published record first, search with floats, certify with integers, ship a standalone verifier — can move a fifty-year-old quantity whose record had not changed since 2009.

References

[1] V. Chvátal and D. Sankoff. Longest common subsequences of two random sequences. *Journal of Applied Probability*, 12(2):306–315, 1975.

[2] J. M. Steele. An Efron–Stein inequality for nonsymmetric statistics. *Annals of Statistics*, 14(2):753–758, 1986.

[3] J. Boutet de Monvel. Extensive simulations for longest common subsequences. *European Physical Journal B*, 7:293–308, 1999.

[4] G. S. Lueker. Improved bounds on the average length of longest common subsequences. *Journal of the ACM*, 56(3):1–38, 2009.

[5] G. T. Heineman, C. Miller, D. Reichman, A. Salls, G. N. Sárközy, and D. Soiffer. Improved lower bounds on the expected length of longest common subsequences. arXiv:2407.10925, 2024.